

A Code Coverage-Based Test Suite Reduction and Prioritization Framework

Saif Ur Rehman Khan, Sai Peck Lee

Faculty of Computer Science and IT
University of Malaya
Kuala Lumpur, Malaysia
saif_rehman@siswa.um.edu.my
saipeck@um.edu.my

Reza Meimandi Parizi

School of Computing and IT
Taylor's University
Selangor, Malaysia
rezaimeimandi.parizi@taylors.edu.my

Manzoor Elahi

Dept. of Computer Science
COMSATS Institute of IT
Islamabad, Pakistan
tamimy@comsats.edu.pk

Abstract—Software testing is extensively used to ensure the development of a quality software system. The test suite size tends to increase by including new test cases due to software evolution. Consequently, the entire test suite cannot be executed considering budget and time limitations. Researchers have examined test suite reduction and prioritization techniques to address the test suite size problem. However, combination of these techniques can be useful for various regression testing situations. In this paper, we present a new code coverage-based test suite reduction and prioritization framework called *TestOptimizer*. The framework performs a suitable combination of *TestFilter* and *St-Total* techniques to determine optimal test cases, keeping in view of time restrictions. The performance of the proposed framework has been assessed using a case study. Results show that *TestOptimizer* can be beneficial to solve the test suite size problem within time constraints and has a profound impact on the required cost and effort of regression testing.

Keywords—regression testing; framework; test suite reduction; test suite prioritization.

I. INTRODUCTION

Software-development organizations greatly depend on testing to make sure the delivery of a quality software system [1]. Regression testing plays an important role in assuring the quality of evolving software systems. However, as software modifies, the test suite size significantly increases by adding new test cases. Furthermore, high human interventions are required to prepare the test-bed or check the test results due to semi-automatic Oracle. Consequently, the entire test suite cannot be executed within budget and time constraints [2].

To solve the test suite size problem, researchers have examined the use of test suite reduction and prioritization techniques [3]. Test suite reduction techniques (e.g. [4][5][6]) aims at reducing the test suite size by removing the redundant test cases from original test suite based on certain coverage requirement [2]. Conversely, test suite prioritization techniques (e.g. [8][9]) focus on ordering the test cases to achieve a certain objective (i.e. fault-detection rate) [2].

Prior work has separately investigated the test suite reduction and prioritization techniques to decrease the regression testing cost. However, combination of these regression testing techniques can be useful in resource

constrained testing situations. For example, test leader needs to know the execution order of the reduced suite to successfully manage the risk of under budget constraint's testing. This goal can be accomplished by first determining the minimized suite using test suite reduction techniques and then by computing the execution order using test prioritization techniques [3].

In this paper, we present a new code coverage-based test suite reduction and prioritization framework called *TestOptimizer* that automatically determines representative and/or high-order test cases based on the allocated test optimization time. We also evaluate the performance of the proposed framework using a case study. The study concludes that *TestOptimizer* is practical in finding a reduced and/or prioritized set of test cases from a large test suite by considering time constraints. Ultimately, it is helpful in decreasing the regression testing cost and effort.

The remainder of this paper is organized as follows: Next Section provides background of coverage-based test suite reduction and prioritization techniques. Furthermore, it provides an overview of related work. Section III presents the proposed framework, while Section IV provides our empirical assessment and case scenarios. Section V presents threats to validity, while Section VI provides the best-practice guidelines useful for different regression testing situations. Finally, the conclusion and future work are presented in Section VII.

II. BACKGROUND AND RELATED WORK

Coverage-based test suite reduction and prioritization techniques optimize test cases based on the achieved coverage of different aspects (e.g. source code or model) of System Under Test (SUT). The code coverage is used to measure the degree of SUT's code exercised by the generated test suite [4]. Furthermore, it provides feedback about the strategy that should be used to enhance the achieved coverage [5]. The coverage criteria (e.g. statement, branch, or path) act as a stopping point to decide, whether the SUT is sufficiently tested or not [4].

A. Test Suite Reduction Techniques

Test Suite Reduction (TSR) techniques focus on finding a *Representative Set* (RS) that attempts to retain the

effectiveness similar to the test suite [5]. The optimal TSR problem can be formally defined as [6]:

Given: A test suite, T , a set of test requirements $\{R_1, R_2, \dots, R_n\}$, that must be satisfied to provide the desired ‘sufficient’ testing of the program, and subsets of T , $\{T_1, T_2, T_3, \dots, T_m\}$, one associated with each of the R_i s such that any one of the test cases t_j belonging to T_i can be used to test a requirement R_i .

Problem: Find a representative set, RS , of test cases from T that satisfies all R_i s.

A significant number of code coverage-based TSR techniques have been proposed (e.g. [4][5][6][15]) to find the RS . The critical threat associated with any TSR technique is the removal of a fault-exposing test case(s). However, Rothermel et al. study [12] concludes that the impact of this risk is so negligible compared to the benefits (e.g. less time in executing the RS , managing the test data, and assessing the test results) using TSR techniques.

B. Test Suite Prioritization Techniques

Test Suite Prioritization (TSP) techniques aim at finding crucial test cases according to some coverage criteria (e.g. fault coverage). As a result, TSP techniques help in improving the fault-detection rate ([5][14]). Certainly, enhanced fault-detection rate provides reliable and earlier feedback regarding the SUT. Thereby, helps the test leader in fixing the faults in a more systematic manner [8]. The optimal TSP problem can be stated as [9]:

Given: A test suite, T , a set of permutations of T , PT , and a function g from PT to real numbers, $g: PT \rightarrow R$.

Problem: Find $RS \in PT$, such that

$$(\forall RS') (RS' \in PT) (RS' \neq RS) [g(RS) \geq g(RS')]$$

In total, 14 techniques have been proposed for test ordering [8]. The TSP techniques can be useful in situations, when the test leader wants to avoid fault-detection loss. Consequently, the ordered test cases can be run until the assigned test execution time expires [9].

Some work has focused on combining TSR and TSP techniques. Bertolino et al. [7] introduced the strategy of ordering the RS to meet the budget constraint testing. Later, Sampath et al. [10] investigate this strategy for web application’s domain. However, prior work did not consider other potential combinations of TSR and TSP techniques, which can be useful for different regression testing situations. To the best of our knowledge, no generalized framework has been proposed that can be beneficial for testing any type of application and also combine TSR and TSP techniques in other possible ways.

III. THE PROPOSED FRAMEWORK

The proposed framework focuses on efficient test case management and helps the test leader in determining optimal test cases by employing TSR and TSP techniques. Note that by optimal, we mean selection of

representative/high-order test cases that can satisfy user requirements and provide the same level of testing effectiveness as achieved by their original counterpart. The TestOptimizer is a generalized framework which can be suitable for any type of application. It just requires a prior knowledge about test-code coverage information (i.e. which portion of SUT is covered by a particular test case). After that, it automatically audits a large pool of test cases according to the allotted optimization time. Figure 1 depicts the high level architecture of TestOptimizer, which contains two main components: (i) Test Planner and (ii) Test Manager. In the following, we briefly discuss these components:

(1) *Test Planner*: It takes inputs from the test leader, which is necessary for correct working of the proposed framework. The required inputs are:

- *Allotted Optimization Time*: The test leader provides it keeping in view of available time for test case execution and related data management. The test leader can compute the optimization time by considering the overall running-time of the employed TSR or TSP technique(s).
- *Test Suites*: It contains information about associated testing requirements and test cases.
- *Test Repository*: It contains necessary information, which may be forwarded to *Test Manager* for effective functioning of TestOptimizer. Note that we have collected the code coverage information using the coverage tool of Tikir et al. [13].

(2) *Test Manager*: It generates optimal test cases using coverage-based TSR and TSP techniques keeping in view of allotted optimization time. Finally, the computed results are saved in the *Test Repository*. The main sub-components are:

- *Optimizer*: It monitors and controls the test optimization process by calling *Reducer* and/or *Prioritizer* (for test reduction and prioritization, respectively) using the saved Test Repository’s information.
- *Timer*: It informs the *Optimizer* about the remaining optimization time after the completion of optimization iteration. Based on this information, *Optimizer* decides which optimization technique should be used for the next iteration and when to stop?

A. Overview of TestOptimizer Functioning

First of all, *Test Planner* accepts required inputs from the test leader. Next, it collects the test-code coverage information and saves the results in the *Test Repository* for regression testing. After that, all computed information is forwarded to the *Test Manager*. Next, *Test Manager* transfers control to the *Optimizer*.

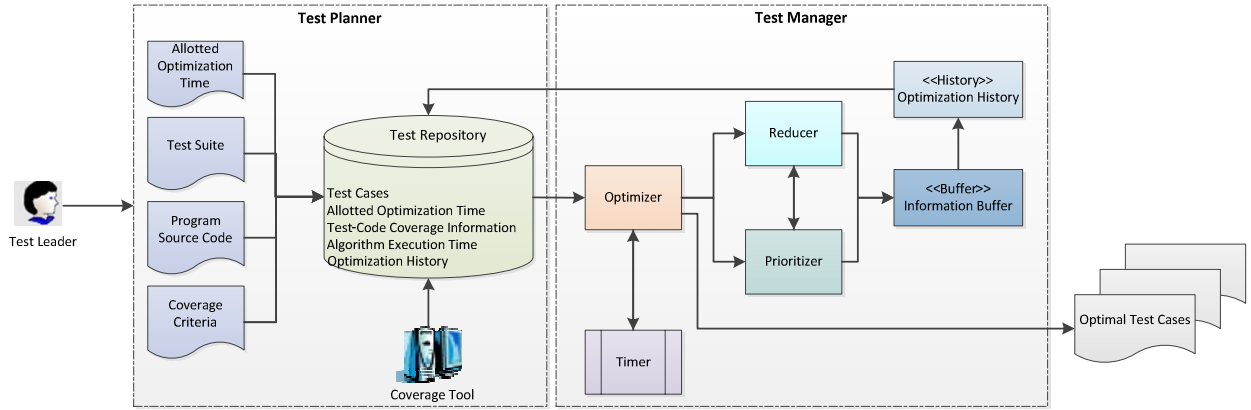


Figure 1. High Level Architecture of TestOptimizer.

The *Optimizer* set the *Allotted Optimization Time* in *Timer* and selects an optimization technique or their combination, which is feasible in the given time. The *Optimizer* can select the following combinations of TSR and TSP techniques by calling *Reducer* and/or *Prioritizer* based on the allocated optimization time:

- (i) Reducer
- (ii) Prioritizer
- (iii) Reducer and then Prioritizer
- (iv) Prioritizer and then Reducer

In the aforementioned combination (iii), *Reducer* first determines the RS and then *Prioritizer* ranks them. So this combination helps the test leader in determining the execution order of representative test cases [3]. Conversely, in combination (iv), *Prioritizer* selects first N number of test cases and then *Reducer* computes the RS from this prioritized set. As a result, combination (iv) helps in improving the fault-detection rate [3]. After execution of each technique, TestOptimizer saves the optimization result in the *Test Repository*. Finally, *Optimizer* generates the optimal test cases. Note that we have employed *TestFilter* technique [4] and *st-total* technique [8] for TSR and TSP, respectively.

B. Assumptions

This section lists the main assumptions necessary for correct functioning of the *TestOptimizer*.

A1: Any test case (t_i) contained in a test suite (T_i) can independently test the associated requirement (R_i).

A2: Test leader can employ any proposed TSR and TSP technique for basic working of *Reducer* and *Prioritizer*, respectively.

IV. EMPIRICAL ASSESSMENT

Proposing a new framework must be accompanied by an evaluation and analysis of the performance before it can be considered for adoption. This section hence describes an experiment conducted to evaluate the performance of TestOptimizer.

A. Overview of Case Study

We used classical Triangle program [11] for empirical evaluation of the TestOptimizer. This program takes three positive integer values as inputs from the user, which represents the lengths of three sides of a triangle. The main purpose is to show a message about type of a triangle: (i) *Isosceles* (i.e. any two sides are equal), (ii) *Scalene* (i.e. no two sides are equal), or (iii) *Equilateral* (i.e. all sides are equal) [11].

The total running time of *Reducer* using *TestFilter* technique is $O(m * n + n (m * n))$ [4] and *Prioritizer* by using *St-Total* technique is $O(n * m + n \log n)$ [8]. In this experimental study, we have used total 49 test cases (i.e. n) and 13 test suites (i.e. m) representing statements of Triangle program (Table I). The test leader can compute the time units required for technique selection by putting the n and m values in their overall running time. Therefore, in this case, the minimum time required for both *Reducer* and *Prioritizer* are 31,850 and 720 time units, respectively. Similarly, test leader can inputs following minimum optimization time units to initiate the optimization process:

- For Reducer: $T_1(n) = 31,850$ time units
- For Prioritizer: $T_2(n) = 720$ time units
- For Reducer and Prioritizer: $T_3(n) = 32,570$ time units
- For Prioritizer and Reducer: $T_4(n) = 32,570$ time units

B. Case Scenarios

This section presents possible case scenarios that help in understanding the working of TestOptimizer. The proposed framework selects TSR and TSP techniques based on the *Allotted Optimization Time*. Therefore, each case scenario represents the time given by the test leader for initiating the test optimization process.

• Case Scenario 1: W Time Units

Suppose the test leader gives W time units (i.e. $T(Reducer) < W < T(Prioritizer)$) as *Allotted Optimization Time*. Note that $T(Reducer)$ and $T(Prioritizer)$ represent the time required for *Reducer* and *Prioritizer*, respectively.

As this is the first version of the program, so control is primarily handed over to the *Test Manager*. Next, the *Optimizer* retrieves necessary information from the *Test Repository* and also informs the *Timer* about the *allotted optimization time*, i.e. 32000 time units ($W - T_1(n)$ or 32000 - 31850). As the given time is sufficient for the working of *Reducer*, so *Optimizer* then calls the *Reducer* to start its functioning.

Initially, the RS is empty. The *TestFilter* (i.e. employed TSR technique) computes the Weighted Set (WS) of all test cases by calculating their membership in the total number of test suites [4]. The weight of t_{29} is 9 (as test case t_{29} is a member of nine test suites); t_{26} to t_{28} , t_{30} to t_{39} , and t_{49} are 8; and remaining test case's weight is 7. So, *TestFilter* picks t_{29} and inserts into the RS (i.e. $\{t_{29}\}$). Next, it marks test suites (i.e. $T_1, T_2, T_3, T_4, T_5, T_6, T_7, T_9$, and T_{11}) as satisfied and eliminates them from Set of Test Suites (STS). After that, *TestFilter* decrement the weight (by one) of all those test cases, which are in the domain of marked test suites. Now the weight of t_{10} to t_{49} converts to zero due to elimination of related test suites from STS. Subsequently, *TestFilter* computes the weights of all other test cases which are in the domain of unmarked test suites. The new WS is:

- $t_1 - t_9 : 7 - 6 = 1$
- $t_{10} - t_{25} : 7 - 7 = 0$
- $t_{26} - t_{28} : 8 - 8 = 0$
- $t_{30} - t_{39} : 8 - 8 = 0$
- $t_{40} - t_{48} : 7 - 7 = 0$
- $t_{49} : 8 - 8 = 0$

Next, it selects test cases having weight more than zero. As t_1 is the first maximum weighted test case (in the aforementioned WS), so it is selected and inserted into the RS (i.e. $\{t_{29}, t_1\}$). Then, *TestFilter* marks and eliminates test suite T_{13} from the STS. Afterwards, *TestFilter* repeats the whole process and decrements the weight of test cases (i.e. t_2, t_3, t_4, t_5, t_6) by one, which are in the domain of the test suite T_{13} . Then, *TestFilter* recalculate the WS of remaining tests. So the new WS of remaining test cases is:

- $t_1 - t_6 : 1 - 1 = 0$
- $t_7, t_9 = 1$

The *TestFilter* selects t_7 and inserts into the RS (i.e. $\{t_{29}, t_1, t_7\}$). After that, *TestFilter* marks and eliminates T_8 from the STS. Next, *TestFilter* computes the weight of test cases, which are in the domain of unmarked test suites. Now two test cases (i.e. t_8 and t_9) with similar weight are left. After that, *TestFilter* randomly selects t_8 and inserts it into the RS (i.e. $\{t_{29}, t_1, t_7, t_8\}$). The *TestFilter* then marks and excludes T_{10} from the STS. Similarly, it picks t_9 and inserts it into the RS (i.e. $\{t_{29}, t_1, t_7, t_8, t_9\}$).

The *TestFilter* then marks and removes T_{12} from the STS. Since all test cases of weight greater or equal to one have been processed. Furthermore, all members of STS have been marked as satisfied. So this ends the TSR process with the final computed RS $\{t_{29}, t_1, t_7, t_8, t_9\}$.

TABLE I. THE TRIANGLE PROGRAM'S COVERAGE INFORMATION [4].

<i>Triangle Program Statements</i>	<i>Req's (R_i)</i>	<i>Test Suites (T_i)</i>	<i>Test Case (t_j) in T_i</i>
if !(a > 0 && b > 0 && c > 0)	R ₁	T ₁	$t_1 - t_{49}$
cout<<"not a triangle"<<endl;	R ₂	T ₂	$t_{26} - t_{39}$
if !((2*a) < (a+b+c) && (2*b) < (a+b+c) && (2*c) < (a+b+c))	R ₃	T ₃	$t_1 - t_{49}$
cout<<"not a triangle"<<endl;	R ₄	T ₄	$t_{11} - t_{49}$
if (a == b)	R ₅	T ₅	$t_1 - t_{49}$
if (b == c)	R ₆	T ₆	$t_1 - t_{49}$
cout<<"triangle is equilateral"<<endl;	R ₇	T ₇	t_{10}, t_{29}, t_{49}
cout<<"triangle is isosceles"<<endl;	R ₈	T ₈	t_7
if (a == c)	R ₉	T ₉	$t_1 - t_{49}$
cout<<"triangle is isosceles"<<endl;	R ₁₀	T ₁₀	t_8
if (b == c)	R ₁₁	T ₁₁	$t_1 - t_{49}$
cout<<"triangle is isosceles"<<endl;	R ₁₂	T ₁₂	t_9
cout<<"triangle is scalene"<<endl;	R ₁₃	T ₁₃	$t_1, t_2, t_3, t_4, t_5, t_6$

Next, this test optimization information is handed over to the *Information Buffer*, where it passes this information to *Optimization History* and *Optimizer*. After that, the *Optimization History* is saved in the *Test Repository* on a permanent basis.

Meanwhile, the *Optimizer* checks whether another optimization technique can be run according to the allocated optimization time. This time the *Timer* informs *Optimizer* that no other technique can be executed as only 150 time units are left. Finally, the *Optimizer* returns $\{t_{29}, t_1, t_7, t_8, t_9\}$ as optimal test cases as shown in Figure 2.

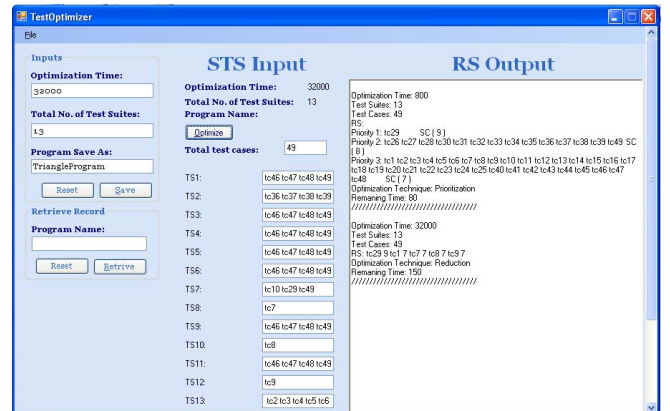


Figure 2. A Snapshot of Graphical User Interface showing the Inputs and Output of TestOptimizer.

- *Case Scenario 2: X Time Units*

Suppose the test leader gives X time units (i.e. $T(Reducer) > X > T(Prioritizer)$) as *Allotted Optimization Time*, i.e. 800 time units ($X - T_2(n)$ or $800 - 720$) sufficient for the working of *Prioritizer*. So in this case, *Optimizer* calls the *Prioritizer* for its functioning.

Table II depicts the identified priority levels using st-total coverage based prioritization technique. It depicts that t_{29} is a high-order test case due to its coverage of nine statements of the Triangle program. In contrast, t_{26} to t_{28} , t_{30} to t_{39} , and t_{49} have second highest priority, and remaining test cases have the least priority. This optimization result is forward to the *Information Buffer*, which then transfers to the *Optimization History* and *Optimizer*. Finally, the *Optimization History* is stored in the *Test Repository*, and *Optimizer* stops the test optimization process due to remaining limited optimization time (i.e. 80 time units).

TABLE II. THE ST-TOTAL BASED PRIORITIZATION.

Priority Level	Statements Covered	Test Case(s)
1	9	t_{29}
2	8	$t_{26} - t_{28}, t_{30} - t_{39}, t_{49}$
3	7	$t_1 - t_{25}, t_{40} - t_{48}$

- *Case Scenario 3: Y Time Units*

Suppose the test leader gives Y time units (i.e. $Y > T(Reducer) \cup T(Prioritizer)$) as *Allotted Optimization Time*. Let *Optimizer* first call the *Reducer* for computation of the RS. The *Reducer* finds RS $\{t_{29}, t_1, t_7, t_8, t_9\}$ as illustrated in Case Scenario 1. Next, *Timer* informs the *Optimizer* that 1150 time units (i.e. $Y - T_1(n)$ or $33000 - 31850$) are remaining, which are sufficient for the working of *Prioritizer*. So *Optimizer* then calls the *Prioritizer* for test optimization. Thus, this combination helps the test leader in determining the execution order of RS.

- *Case Scenario 4: Z Time Units*

Suppose the test leader gives Z time units (i.e. $Z > T(Prioritizer) \cup T(Reducer)$) as *Allotted Optimization Time*. Now this time *Optimizer* first calls *Prioritizer* that identifies the execution order of the given test cases. Then, *Timer* informs the *Optimizer* that 32,480 time units (i.e. $Z - T_2(n)$ or $33200 - 720$) are remaining, which are sufficient for the working of *Reducer*. After that, *Optimizer* calls the *Reducer* for determination of their representative test cases.

This optimization combination first identifies test cases of high-priority using *Prioritizer* and then determines the RS using *Reducer*. The result of *Prioritizer* is depicted in Table II. As t_{29} is a high-priority test case. Thus, *Reducer* picks t_{29} and inserts it into RS (i.e. $\{t_{29}\}$). Then, *TestFilter* determines RS from the next high-priority test cases (i.e. from $t_{26}, t_{27}, t_{28}, t_{49}$, and t_{30} to t_{39}). The *TestFilter* computes the WS of these test cases. Next, *TestFilter* picks t_{26} (i.e. first maximum weighted test case) and adds into the RS (i.e. $\{t_{29}, t_{26}\}$). The *TestFilter* then marks test suites (i.e. T_1 to T_6 ,

T_9 , and T_{11}) as satisfied and excludes them from STS. Next, it picks t_{49} (i.e. $\{t_{29}, t_{26}, t_{49}\}$).

Finally, *TestFilter* marks and removes T_7 from the STS. After that, *Reducer* picks the test cases (i.e. t_1 to t_{25} , and t_{40} to t_{48}) from next category. The *TestFilter* again calculates the weights of these lowest priority test cases. The *TestFilter* then picks t_1 and inserts it into the RS (i.e. $\{t_{29}, t_{26}, t_{49}, t_1\}$). Next, *TestFilter* marks $T_1, T_3, T_5, T_6, T_9, T_{11}$, and T_{13} as satisfied and eliminates them from the STS. After that, *TestFilter* decrements the weight by one of those test cases, which are in the domain of currently eliminated test suites. So the remaining test cases with weight one are t_7 to t_{25} , and t_{40} to t_{48} . Then, it picks t_7 and inserts into the RS (i.e. $\{t_{29}, t_{26}, t_{49}, t_1, t_7\}$).

The *TestFilter* then marks and removes T_8 from the STS. After that, *TestFilter* decrements the weight of those test cases, which are in the domain of T_8 . Since no test case lies within the domain of T_8 , so there will be no impact on the weights of other test cases. Likewise, *TestFilter* computes the WS of remaining test cases. Now, it picks t_8 and adds into the RS (i.e. $\{t_{29}, t_{26}, t_{49}, t_1, t_7, t_8\}$). Then, *TestFilter* decrements the weight of those test cases, which are in the domain of T_{10} . After that, *TestFilter* determines the weight of those test cases that belong to the unmarked test suites. Similarly, *TestFilter* picks t_9 and adds into the RS. The resultant RS is $\{t_{29}, t_{26}, t_{49}, t_1, t_7, t_8, t_9\}$.

The *TestFilter* then marks and excludes T_{12} from the STS. After that, it decrements the weight of those test cases, which are in the domain of T_{12} . Similarly, *TestFilter* picks t_{10} and t_{11} as both are satisfying T_7 and T_4 , respectively. The *TestFilter* marks and eliminates T_7 and T_4 from the STS. The resultant RS is $\{t_{29}, t_{26}, t_{49}, t_1, t_7, t_8, t_9, t_{10}, t_{11}\}$. As all test suites are eliminated from the STS. Consequently, this stops the TSR process. So the final RS is $\{t_{29}, t_{26}, t_{49}, t_1, t_7, t_8, t_9, t_{10}, t_{11}\}$.

V. THREATS TO VALIDITY

The major threat to the external validity of our results is that we consider a small size case study. Experiments with larger size of subjects may produce different results. However, we consider a classical Triangle program as a case study to show the combination of TSR and TSP suitable for various regression testing situations. Therefore, we still have the same combination of TSR and TSP irrespective of the size of subjects.

Threats to internal validity involve the generated test cases that can bias our results. In order to mitigate this threat, we have manually used different black box and white box test techniques and carefully developed the test cases for program subject.

VI. DISCUSSION

Based on the presented results, this section discusses the possible phenomena that could explain the trends observed in analysis, as well as relevant points in relation to this area of research.

Both Test Suite Reduction (TSR) and Test Suite Prioritization (TSP) techniques play a vital role in improving the regression testing cost [3]. The test leader can take advantage of these regression test optimization techniques by combining them in such a way that is effective for the considered regression testing situations.

We have identified some testing scenarios that act as best-practice guidelines, where the test leader can take maximal advantage of these optimization techniques. The identified testing scenarios are:

- If a test leader wants to avoid any fault-detection capability loss (i.e. for safety critical systems). The test leader must employ TSP technique.
- If a test leader wants to improve the fault-detection rate by running most important tests within available test execution time. Then, test leader must use TSP technique.
- If a test leader has limited testing resources and marginal loss in fault-detection is acceptable. Then, TSR technique provides a viable solution for this situation.
- If a test leader wants to know the execution order of the Representative Set (RS), so that high-priority tests should be executed first for a budget-constraint testing situation. In this case, the test leader should initially employ TSR technique and then TSP technique.
- If a test leader wants to manage both test optimization and execution process in a parallel manner, the obvious combination is TSP and then TSR. In this combination, test leader first divides the test cases into various levels based on their achieved coverage. Next, pick the high-priority test cases (i.e. N test cases) from top level using TSP technique and then identifies RS from this level. This process continues in a similar manner until all defined levels have been optimized.

VII. CONCLUSION AND FUTURE WORK

Test suite reduction and prioritization techniques attempt to decrease the regression testing cost and effort. This paper presented a novel code coverage-based framework TestOptimizer, which automatically generates crucial test cases considering the allocated test optimization time. Furthermore, the paper presented the results of an empirical study conducted to evaluate the performance of the proposed framework. The results indicated that the TestOptimizer can effectively reduce and/or prioritize the test suite without losing its code coverage score. The results also led to the successful identification of various case scenarios, which consequently helped us in providing best-practice guidelines to use test suite reduction and prioritization techniques for test optimization.

In future, we plan to conduct additional case studies of varying nature to further assess the framework performance and scalability.

ACKNOWLEDGMENT

S.U.R. Khan and S.P. Lee's work is supported by University of Malaya, High Impact Research Grant with reference UM.C/625/1/HIR/MOHE/FCSIT/13, funded by the Ministry of Education, Malaysia. R.M. Parizi's work is supported in part by Taylor's University grant with reference TRGS/ERFS/1/2014/SOCIT/006. The corresponding author would like to acknowledge the financial support of the Bright Sparks Program (BSP) at University of Malaya.

REFERENCES

- [1] W.R. Adrion, M.A. Branstad, and J.C. Cherniavsky, "Validation, Verification, and Testing of Computer Software," *ACM Comp. Survey*, vol. 14, no. 2, June 1982, pp. 159-192, doi:10.1145/356876.356879.
- [2] S. Yoo, and M. Harman, "Regression Testing Minimisation, Selection and Prioritisation: A Survey," *Soft. Test., Verif. and Rel.*, vol. 22, no. 2, March 2012, pp. 67-120, doi:10.1002/stv.430.
- [3] S.U.R. Khan, I. Rehman, and S.U.R. Malik, "The Impact of Test Case Reduction and Prioritization on Software Testing Effectiveness," *Proc. IEEE Int. Conf. Emerging Tech.*, Oct. 2009, pp. 416-421, doi:10.1109/ICET.2009.5353136
- [4] S.U.R. Khan, A. Nadeem, and A. Awais, "TestFilter: A Statement Coverage based Test Case Reduction Technique," *Proc. 10th IEEE Int. Multitopic Conf.*, Dec. 2006, pp. 275-280, doi:10.1109/INMIC.2006.358177
- [5] D. Hao, L. Zhang, X. Wu, H. Mei, and G. Rothermel, "On-Demand Test Suite Reduction," *Proc. 34th Int. Conf. Soft. Eng.*, June 2012, pp. 769-779, doi:10.1109/ICSE.2012.6227144
- [6] M.J. Harrold, R. Gupta, and M.L. Soffa, "A Methodology for Controlling the Size of Test Suite," *ACM Trans. Soft. Eng. Meth.*, vol. 2, no. 3, July 1993, pp. 270-285, doi:10.1145/152388.152391
- [7] A. Bertolino, E. Cartaxo, P. Machado, E. Marchetti, and J.F. Ouiriques, "Test Suite Reduction in Good Order: Comparing Heuristics from a New Viewpoint," *Proc. IEEE Int. Conf. on Testing Soft. and Sys.*, 2010, pp. 13-18. doi:10.13140/2.1.4869.8886 C
- [8] G. Rothermel, R.H. Untch, C. Chengyun, and M.J. Harrold, "Prioritizing Test Cases for Regression Testing," *IEEE Trans. Soft. Eng.*, Sept. 2001, vol. 27, no. 10, pp. 929-948, doi: 10.1109/32.464549.
- [9] J.A. Jones, and M.J. Harrold, "Test-Suite Reduction and Prioritization for Modified Condition/Decision Coverage," *IEEE Trans. Soft. Eng.*, March 2003, vol. 29, no. 3, pp. 195-209, doi:10.1109/TSE.2003.1183927
- [10] S. Sampath, and R.C. Bryce, "Improving the effectiveness of test suite reduction for user-session-based testing of web applications," *Info. and Soft. Tech.*, July 2012, vol. 54, no. 7, pp. 724-738, doi:10.1016/j.infsof.2012.01.007
- [11] G.J. Myers, *The Art of Software Testing*, Wiley-Inter science Publication, New York, 1979.
- [12] G. Rothermel, M.J. Harrold, J.V. Ronne, and C. Hong, "Empirical Studies of Test Suite Reduction," *Soft. Test.Verif. and Rel.*, Dec. 2002, vol. 12, no. 4, pp. 219-249, doi:10.1002/stvr.256
- [13] M.M. Tikir, and J.K. Hollingsworth, "Efficient Instrumentation for Code Coverage," *ACM SIGSOFT Soft. Eng. Notes*, vol. 27, no. 4, July 2002, pp. 86-96, doi:10.1145/566171.566186
- [14] S. Haidry, and T. Miller, "Using Dependency Structures for Prioritization of Functional Test Suites," *IEEE Trans. Soft. Eng.*, vol. 39, no. 2, pp. 258-275, Feb. 2013, doi: 10.1109/TSE.2012.26.
- [15] Q. Mayo, R. Michaels, and R. Bryce, "Test Suite Reduction by Combinatorial-Based Coverage of Event Sequences," *Proc. IEEE Seventh Conf. on Soft. Testing, Verif. and Valid.*, March 2014, pp.128-132, doi:10.1109/ICSTW.2014.14